

Nodejs session 3

▼ Coding

```
const crypto = require("crypto");

process.env.UV_THREADPOOL_SIZE = 5;

const start = performance.now();

crypto.pbkdf2("secret", "salt", 100000, 64, "sha512", (err, derivedKey) => {
  if (err) throw err;
  console.log("Time taken:", performance.now() - start, "ms");
});

crypto.pbkdf2("secret", "salt", 100000, 64, "sha512", (err, derivedKey) => {
  if (err) throw err;
  console.log("Time taken:", performance.now() - start, "ms");
});

crypto.pbkdf2("secret", "salt", 100000, 64, "sha512", (err, derivedKey) => {
  if (err) throw err;
  console.log("Time taken:", performance.now() - start, "ms");
});

crypto.pbkdf2("secret", "salt", 100000, 64, "sha512", (err, derivedKey) => {
  if (err) throw err;
  console.log("Time taken:", performance.now() - start, "ms");
});
```

```

});

crypto.pbkdf2("secret", "salt", 100000, 64, "sha512", (err, derivedKey) => {
  if (err) throw err;
  console.log("Time taken:", performance.now() - start, "ms");
});

fetch("https://dummyjson.com/products").then(() => {
  console.log("Fetch completed", performance.now() - start, "ms");
})
fetch("https://dummyjson.com/products").then(() => {
  console.log("Fetch completed", performance.now() - start, "ms");
})
fetch("https://dummyjson.com/products").then(() => {
  console.log("Fetch completed", performance.now() - start, "ms");
})
fetch("https://dummyjson.com/products").then(() => {
  console.log("Fetch completed", performance.now() - start, "ms");
})
fetch("https://dummyjson.com/products").then(() => {
  console.log("Fetch completed", performance.now() - start, "ms");
})
fetch("https://dummyjson.com/products").then(() => {
  console.log("Fetch completed", performance.now() - start, "ms");
})

```

```

    art, "ms");
  })

  const data = fs.readFileSync("file.txt", "utf-8");
  console.log(data);

  fs.readFile("file.txt", "utf-8", (err, data) => {
    if (err) {
      console.error(err);
    } else {
      console.log(data);
    }
  });
});

const http = require("http");

const server = http.createServer((req, res) => {
  console.log("Received request:", req.url);
  if (req.url === "/") {
    res.end("home bage")
  } else if (req.url === "/about") {
    res.end("about page")
  } else {
    res.statusCode = 404;
    res.end("Page not found");
  }
  res.end("Hello, World!");
});

server.listen(3000, () => {
  console.log("Server is running on http://localhost:3000");
});

```

```
});
```

▼ Explanation

أولاً: الصورة الكبيرة (Node.js يشتغل إزاي؟) 🧠

Node.js يشتغل بـ:

- Thread واحد (Single Thread)

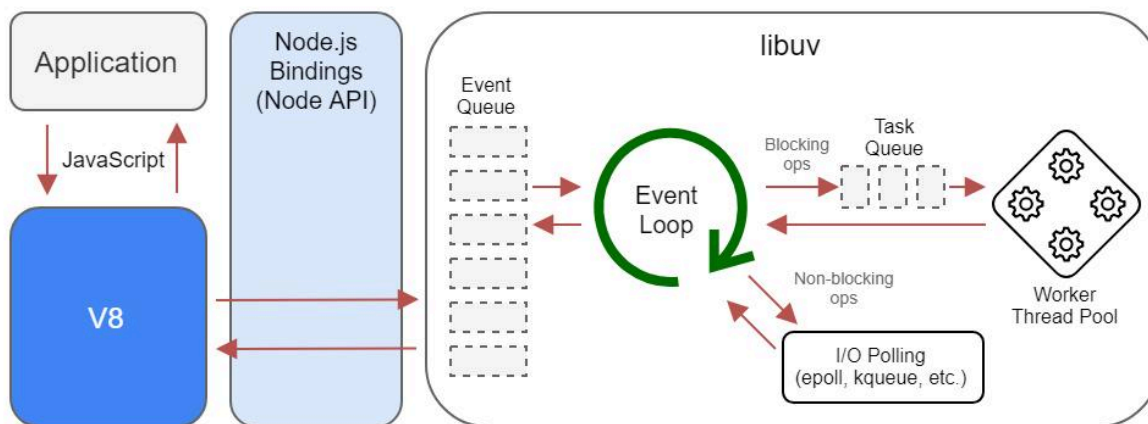
• لكنه بيعمل Async handling باستخدام:

- Event Loop
- Libuv
- Thread Pool

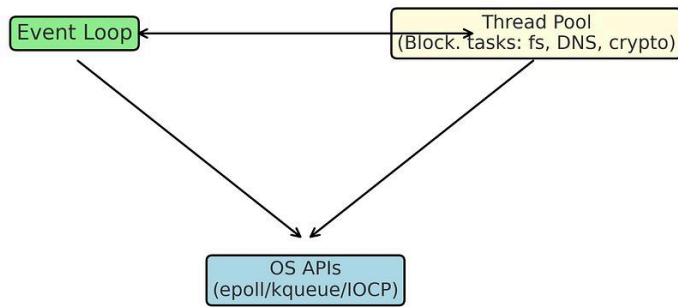
الهدف: 💡

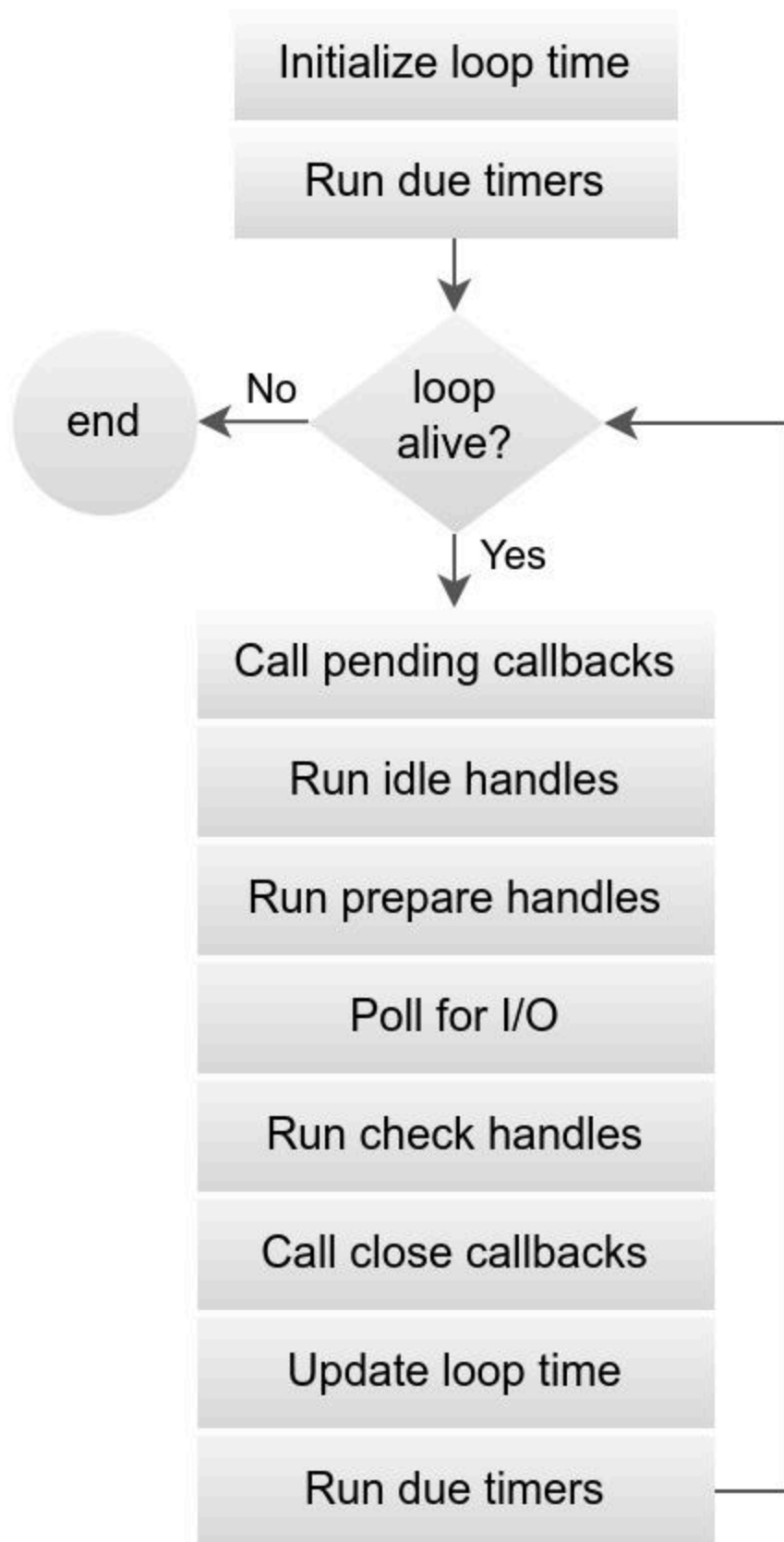
تشغل آلاف ال requests من غير ما البرنامج يهجم

⚙️ 1. Libuv (الحقيقي Node.js قلب)



libuv: Event Loop and Thread Pool





هو إيه؟ 📌

مسؤولة عن C دي مكتبة مكتوبة بـ **Libuv**

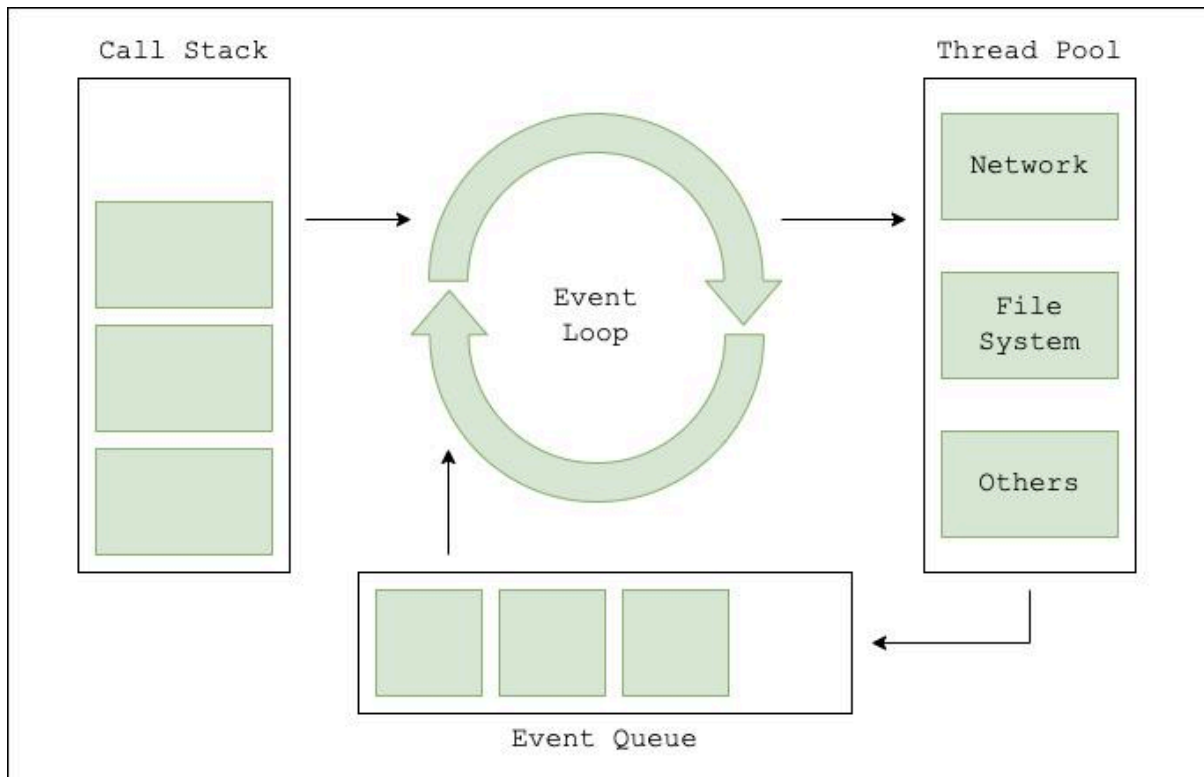
- إدارة الـ async operations
- تشغيل الـ event loop
- التعامل مع الـ OS
- تشغيل thread pool

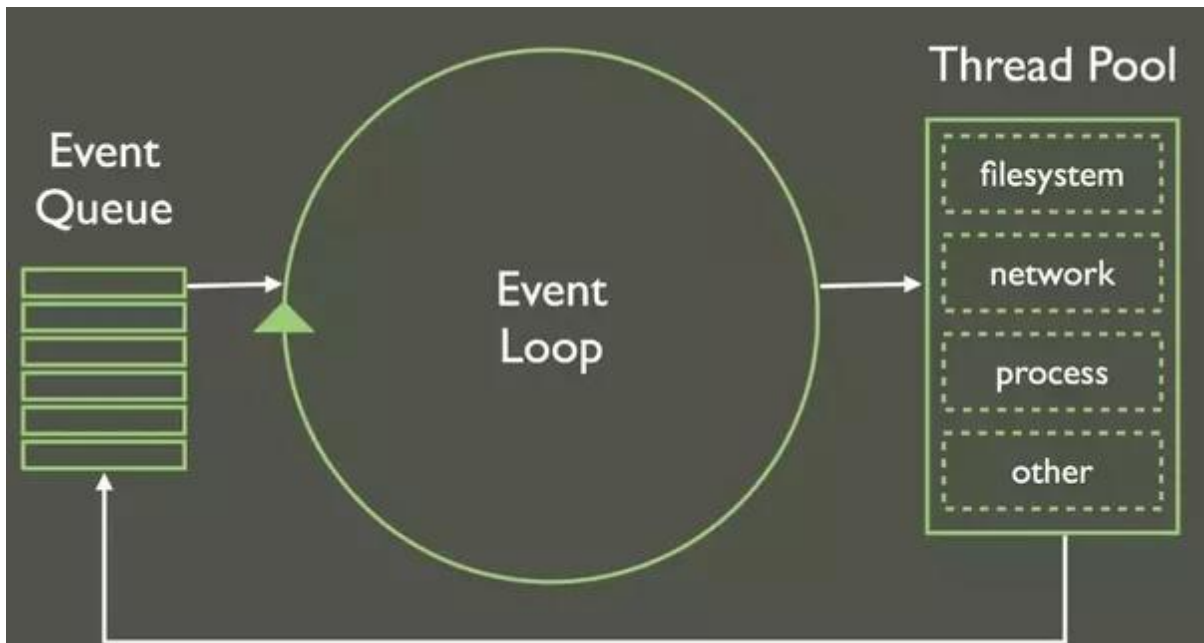
ببساطة: 📌

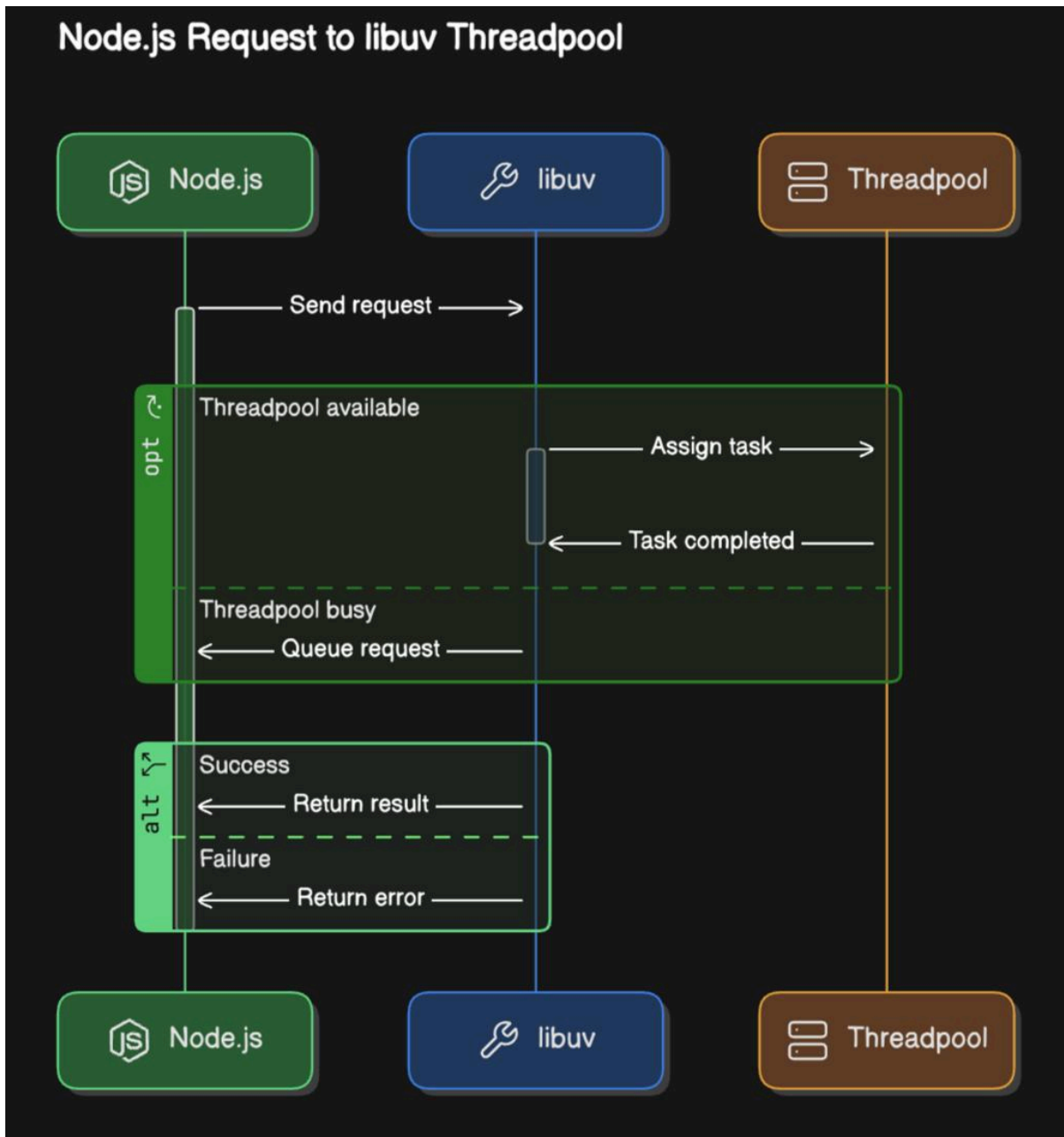
Node.js = JavaScript

لكن التنفيذ الحقيقي **Libuv** =

🧵 2. Thread Pool







6

هو إيه؟ 📌

مجموعة Threads جاهزين ينفذوا tasks ثقيلة بدل ما يوقفوا ال main thread

بيستخدم في: 📌

- bcrypt

- fs (قراءة/كتابة ملفات)
- crypto
- dns

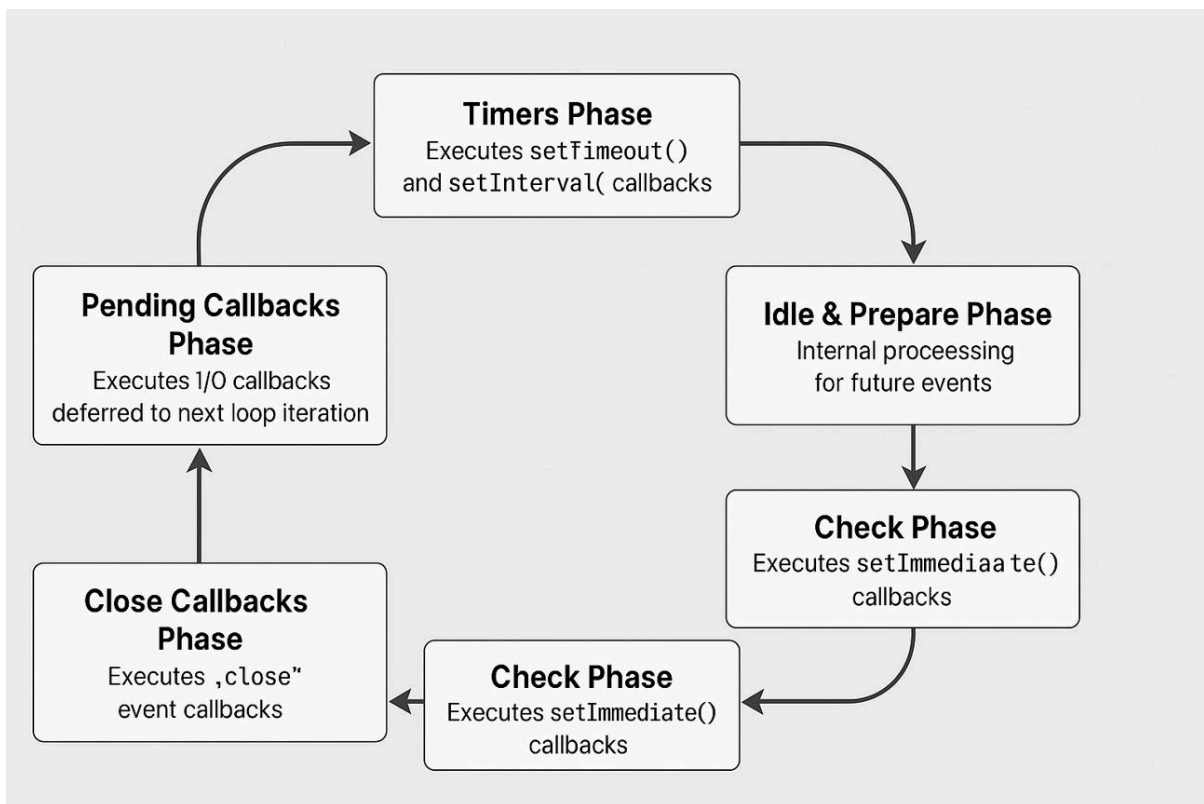
مثال عملي: 📌

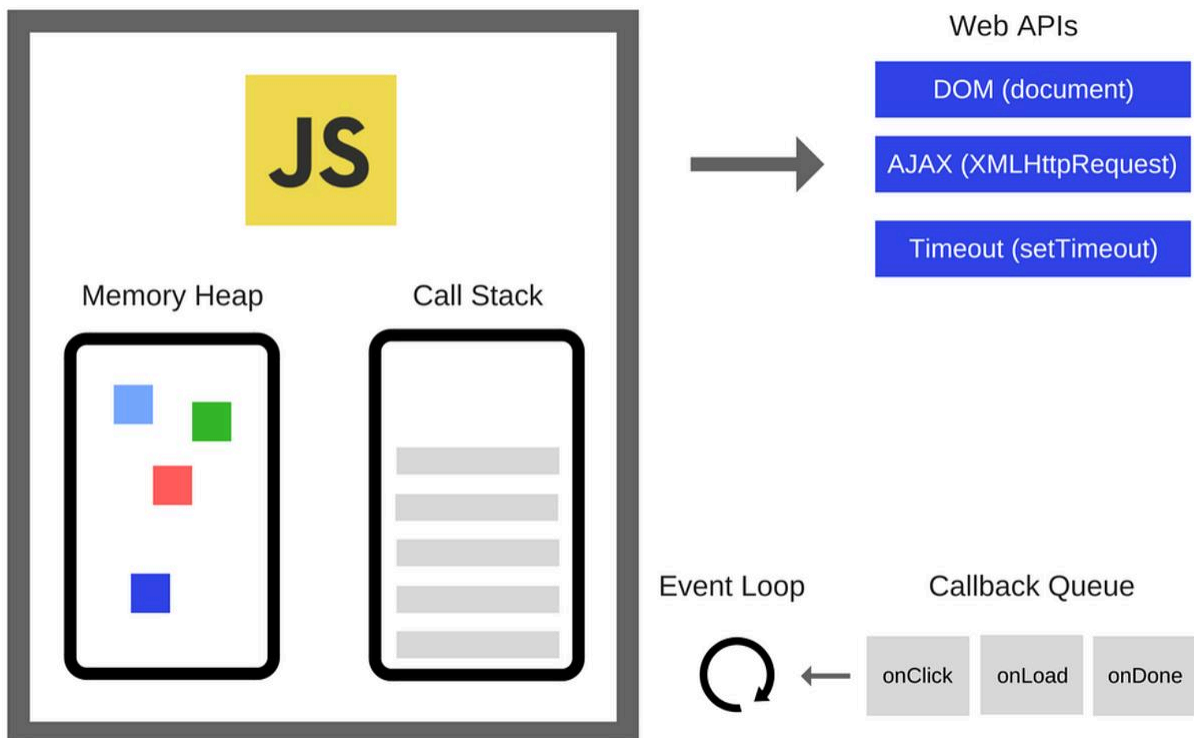
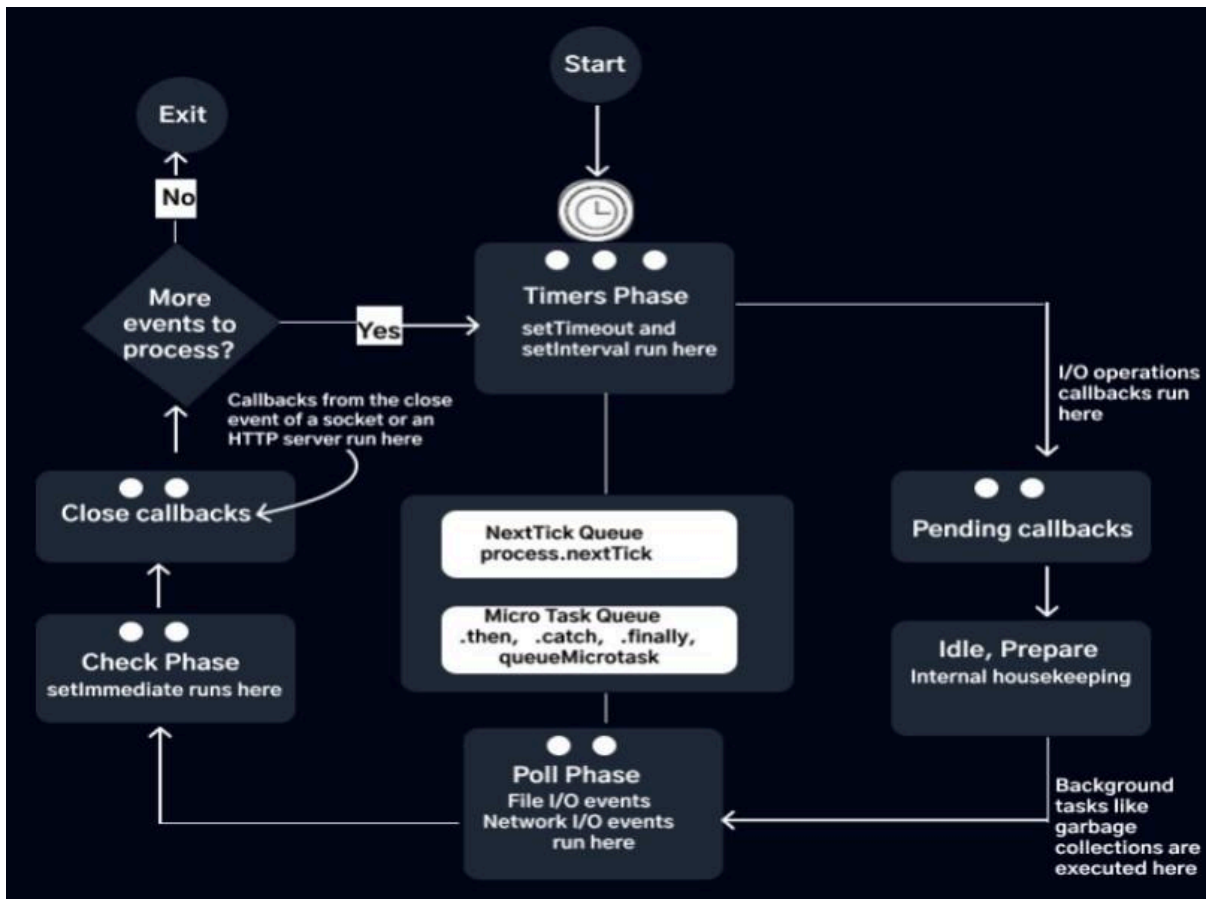
```
const crypto = require('crypto');

crypto.pbkdf2("pass", "salt", 100000, 64, "sha512", () => {
  console.log("Done");
});
```

العملية دي ثقيلة → تروح لل Thread Pool مش لل main thread 💡

🔄 3. Event Loop





6

هو إيه؟ 📌

هو المسؤول عن:

"يشوف إيه جاهز يتنفذ ويشغله"

بيشتغل إزاي؟ 📌

- يلف في loop
- يشوف ال queues
- ينفذ ال callbacks الجاهزة

تخيله كده: 📌

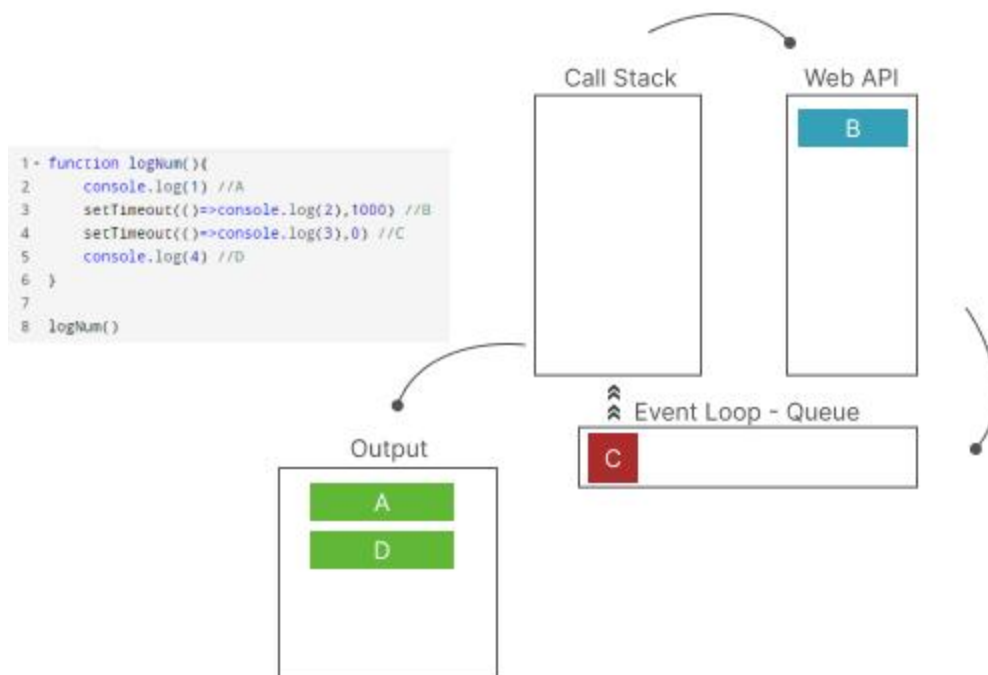
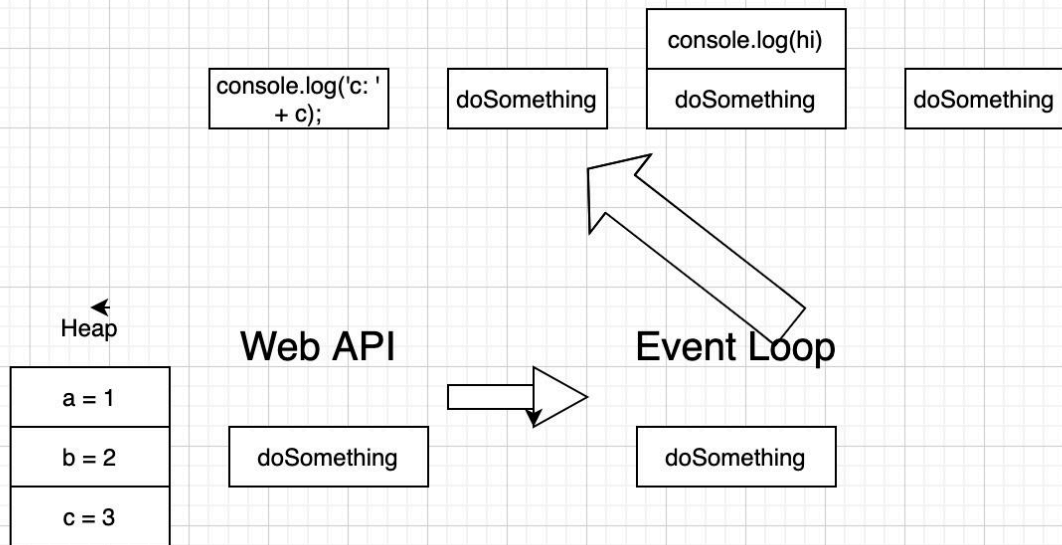
زي مدير مطعم:

- يشوف الطلبات الجاهزة
- يوديها لل waiter

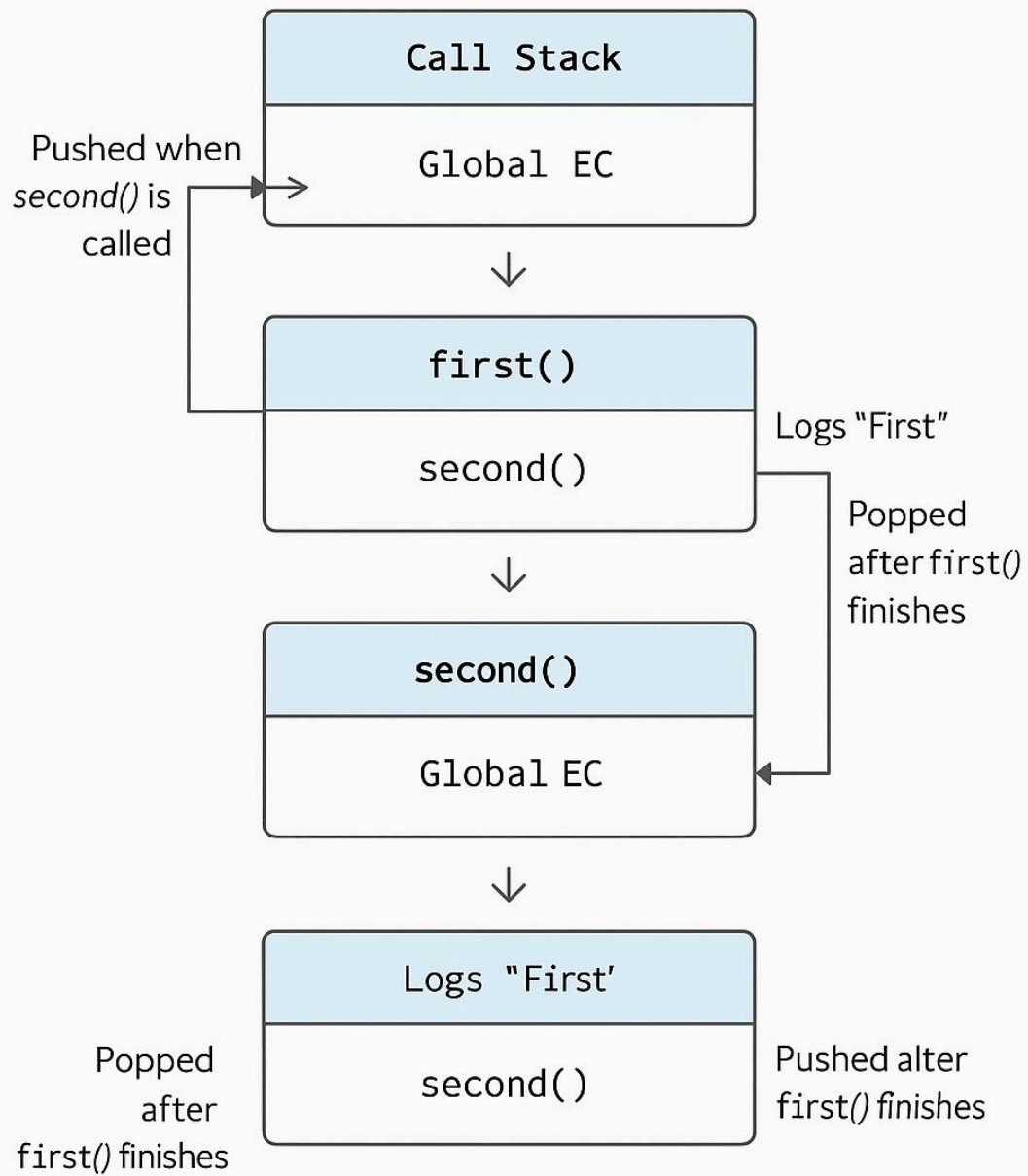


4. Call Stack

Call Stack



```
function first() {
  console.log("First");
}
function second() {
  first();
}
second();
```



5

هو إيه؟ 📌

مكان تنفيذ الكود **synchronous**

مثال: 📌

```
function a() {  
  b();  
}  
function b() {  
  console.log("Hello");  
}  
  
a();
```

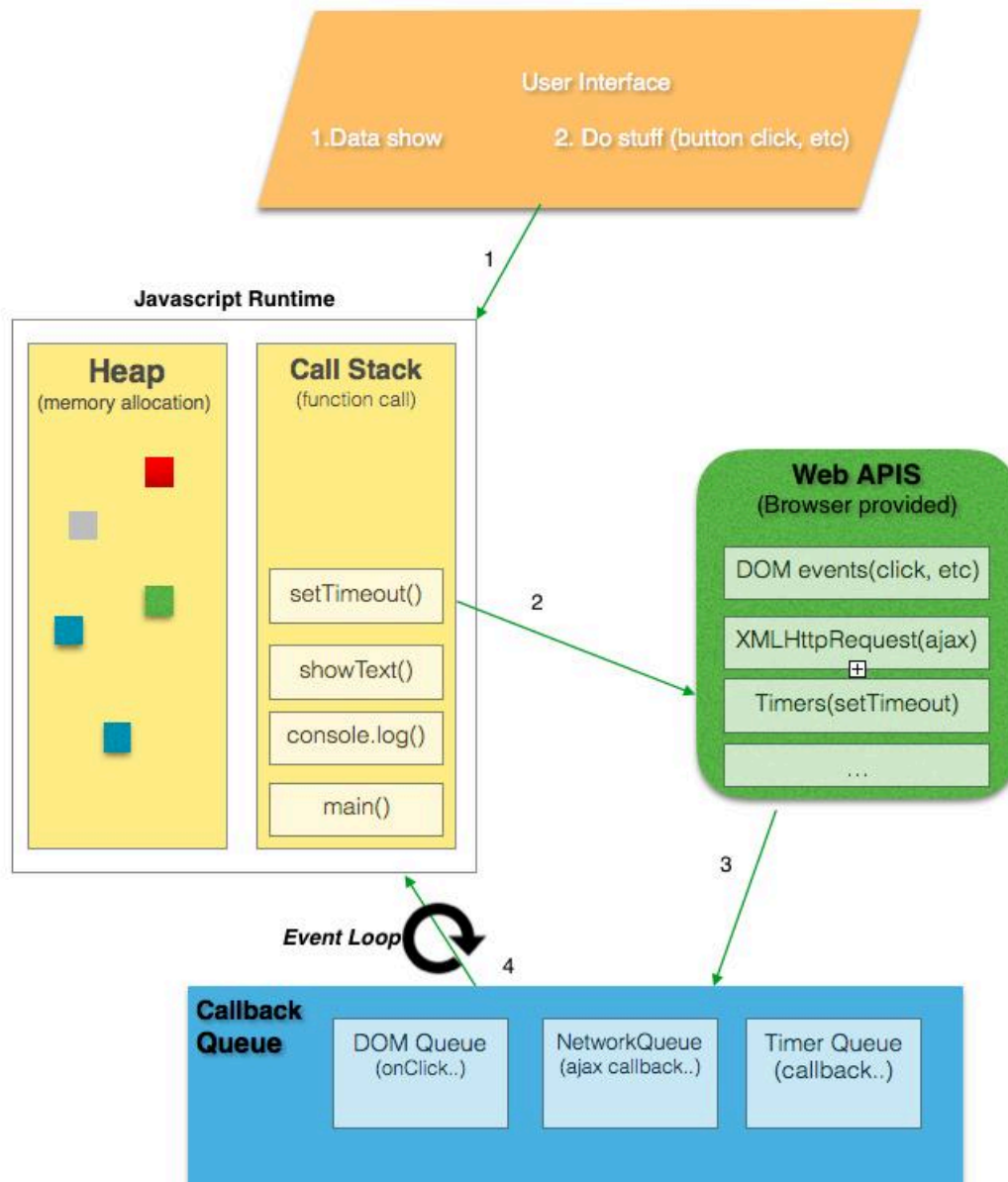
اللي بيحصل: 📌

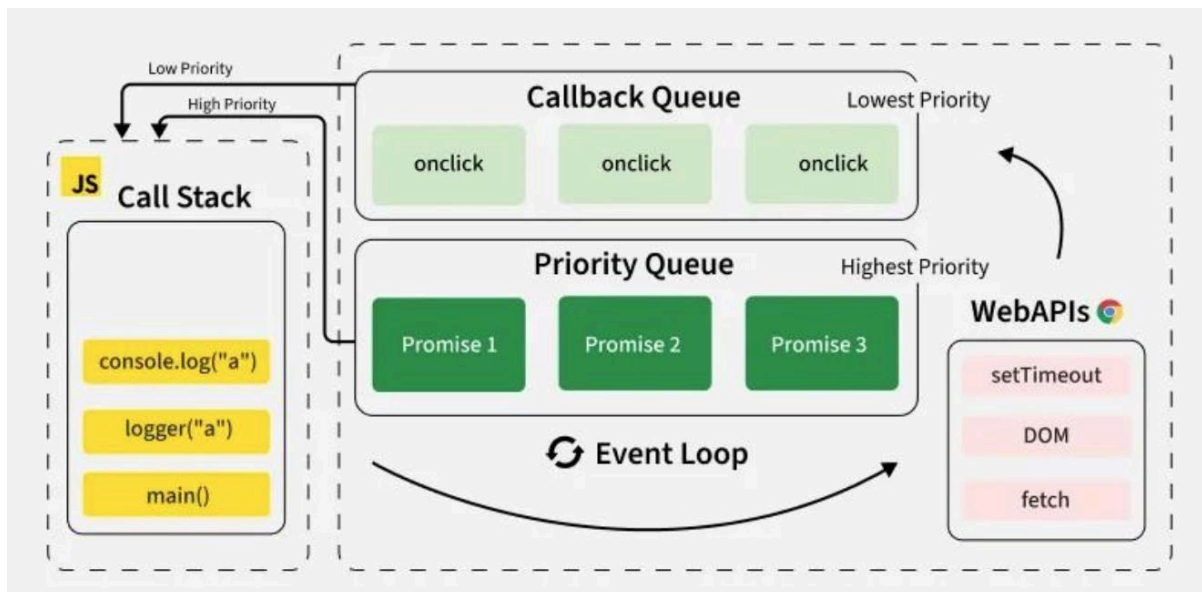
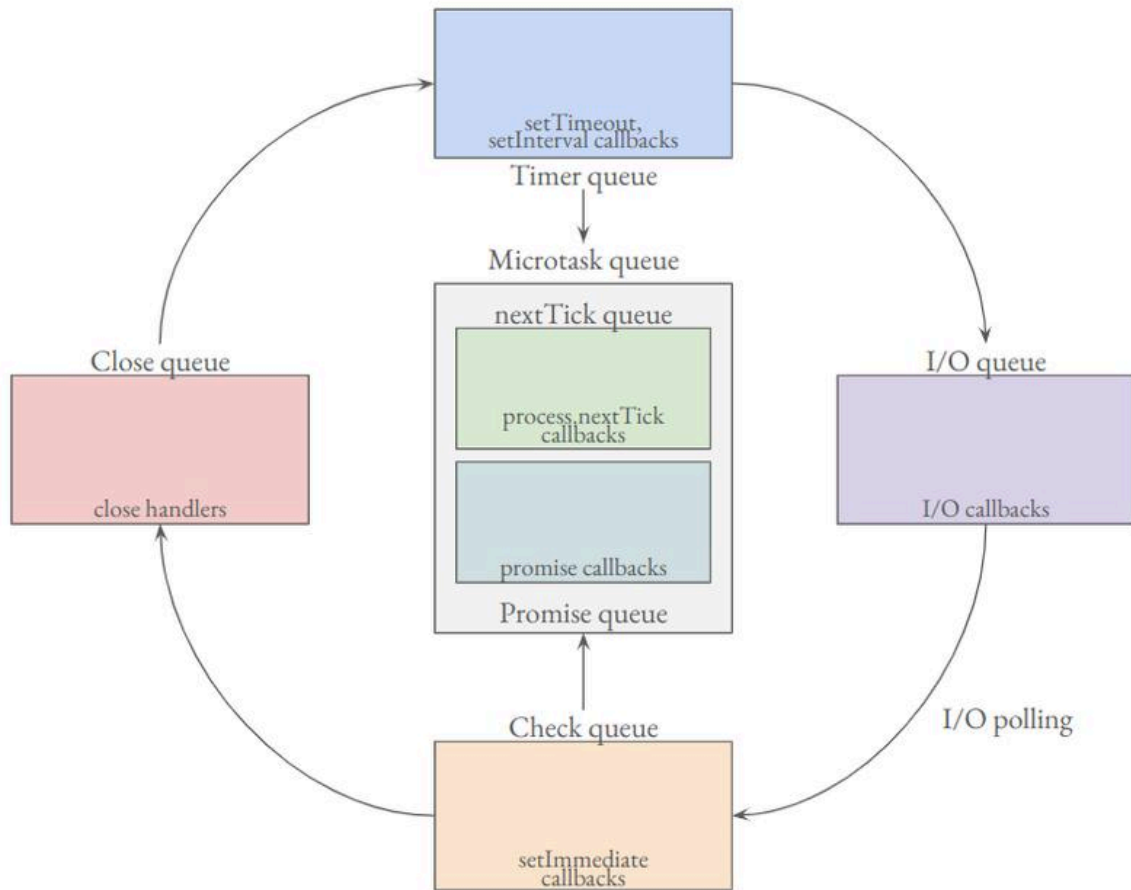
- a تدخل stack
- b تدخل stack
- console.log

• يطلعوا بالترتيب

💡 stack = LIFO (آخر حاجة تدخل تطلع الأول)

📁 5. Callback Queue





هو إيه؟ 📌

Queue async operations لما تخلص callbacks بيتحط فيه ال

مثال: 📌

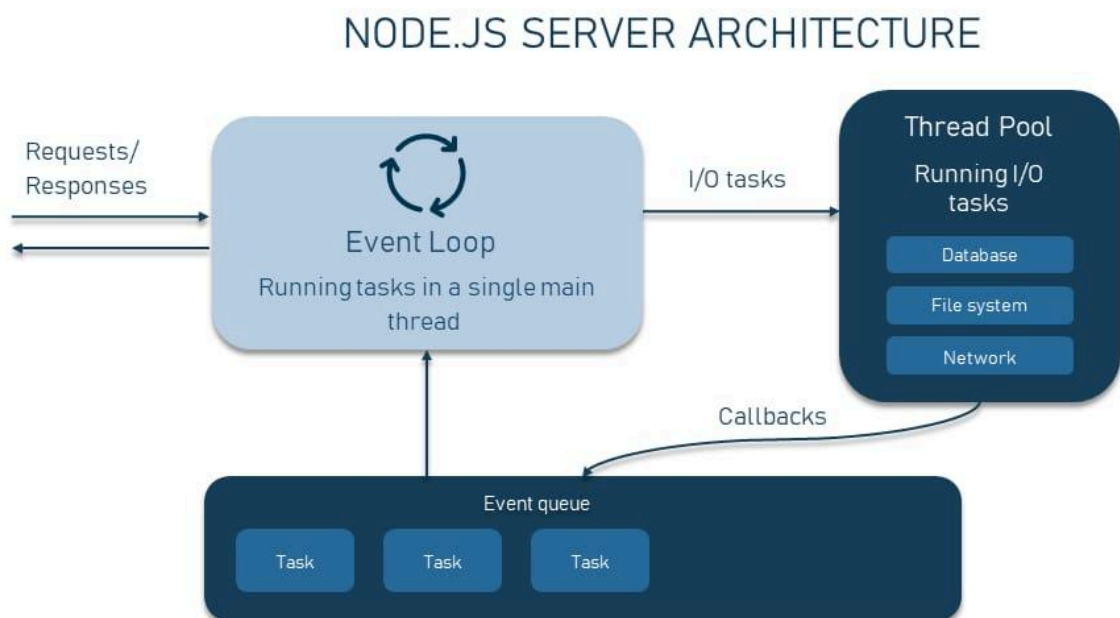
```
setTimeout(() => {  
  console.log("Hi");  
}, 1000);
```

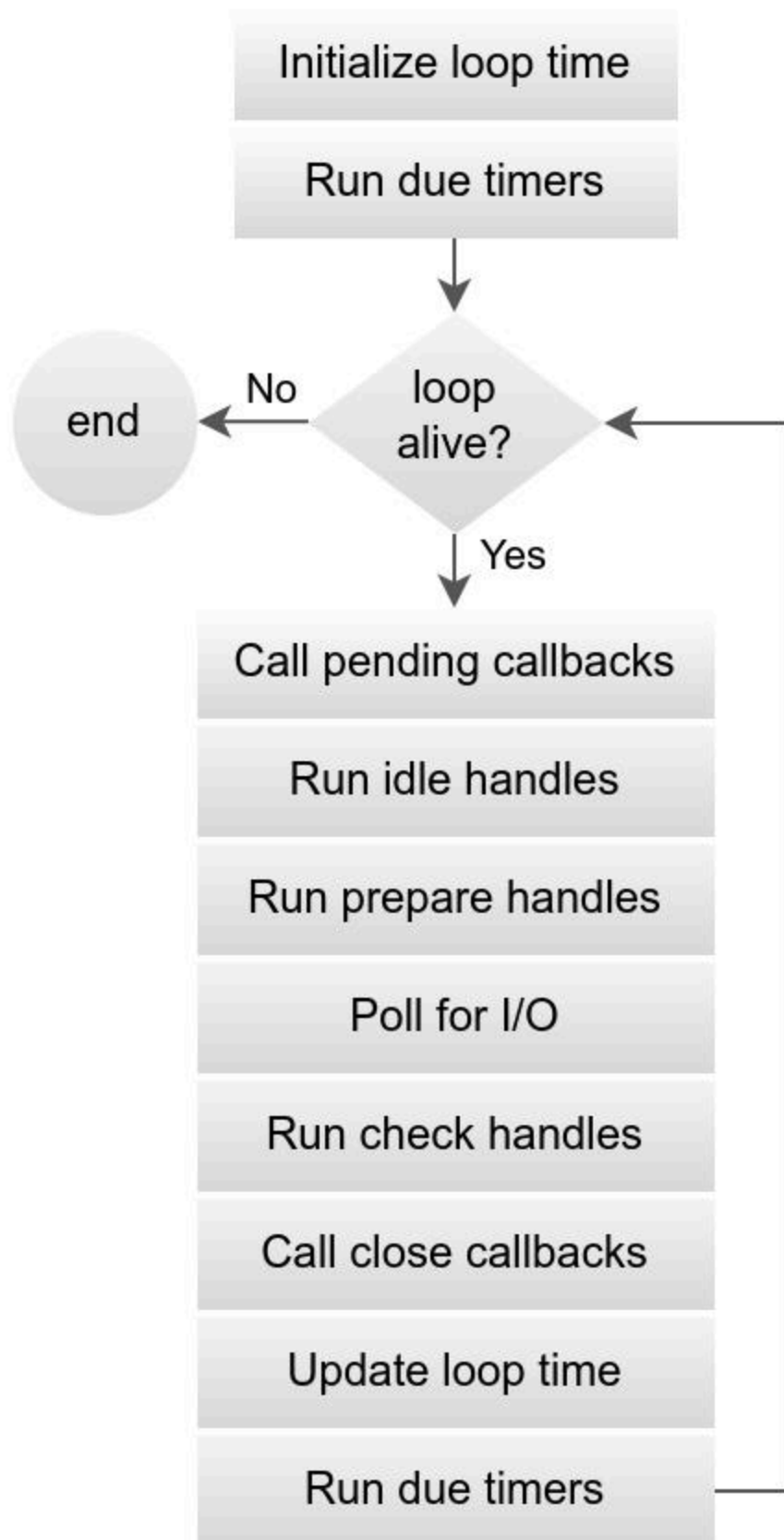
بعد 1 ثانية: 💡

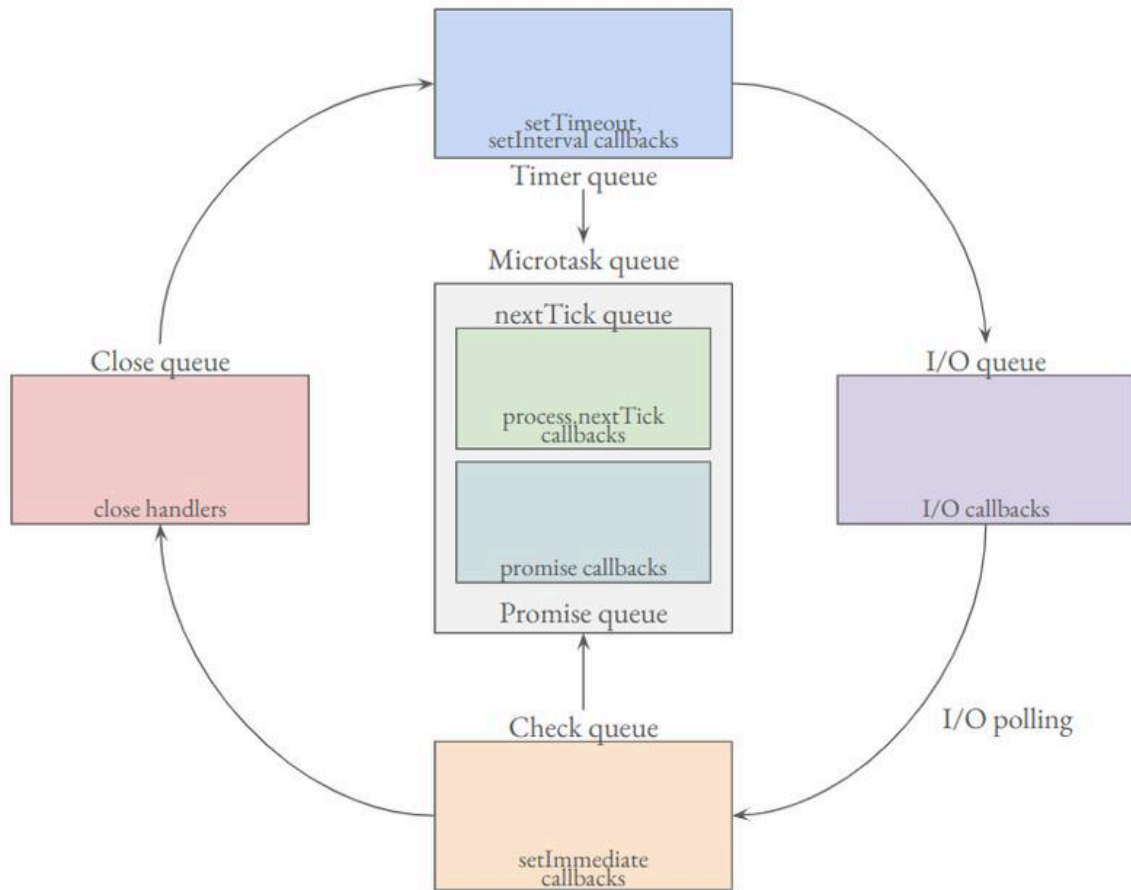
• الكود يتحط في callback queue

- event loop يحطه في call stack

6. العلاقة بين كل دول (flow كامل) 🔄







7

السيناريو: 📌

```
const fs=require('fs');

fs.readFile('file.txt',utf8,() => {
  console.log('Done');
});

console.log('Hello');
```

التنفيذ: 🔥

1. `fs.readFile`
→ يروح لـ Libuv

→ thread pool يتنفذ في

2. `console.log("Hello")`

→ call stack يتنفذ فورًا في

3. لما القراءة تخلص

→ callback يتخط في callback queue

4. event loop

→ يأخذه

→ call stack يحطه في

→ يتنفذ

النتيجة: 📌

```
Hello
Done
```

🧩 7. Callback Phases (مهم جدًا)

ال Event Loop له phases:

1. **Timers** → setTimeout / setInterval

2. **Pending callbacks**

3. **Poll** → I/O (زي fs)

4. **Check** → setImmediate

5. **Close callbacks**

كل phase ليها queue 💡

🌐 8. Create HTTP Server (تطبيق عملي)

```
const http=require('http');
```

```
const server=http.createServer((req,res) => {
  res.end('Hello World');
});

server.listen(3000, () => {
  console.log('Server running');
});
```

اللي بيحصل: 📌

• كل request:

◦ يدخل event loop

◦ يتخط callback

◦ يتنفذ لما الدور عليه

💡 عشان كده Node يقدر يشيل آلاف requests

🧠 الخلاصة (بشكل بسيط جدًا)

العنصر	وظيفته
Libuv	تحت async يشغل كل الـ
Thread Pool	ينفذ العمليات الثقيلة
Event Loop	المدير اللي بينظم التنفيذ
Call Stack	تنفيذ الكود المباشر
Callback Queue	async انتظار الكود

🔥 أهم فكرة لازم تطلع بيها

مش سريع عشان هو قوي Node.js

هو سريع عشان مش بيستنى